

ColumnDetection

August 7, 2024

1 Column Detection on Sample Data

First, we will load in the set of sample tables.

```
[ ]: from torchvision import transforms
import numpy as np
import pandas as pd
import cv2
from PIL import Image

#| export
from huggingface_hub import hf_hub_download
from transformers import AutoModelForObjectDetection
import torch
from PIL import Image
import os
```

```
[ ]: tables_folder = "./sampleTables"
```

```
[ ]: out_folder = "tables"
```

```
[ ]: table_paths = []
for folder in os.listdir(out_folder):
    tpage = os.path.join(out_folder, folder)
    for table in os.listdir(tpage):
        table_paths.append(os.path.join(tpage, table))
```

1.1 Load the model for column detection

```
[ ]: device = "cuda" if torch.cuda.is_available() else "cpu"
model = AutoModelForObjectDetection.from_pretrained("microsoft/
↳table-transformer-structure-recognition", revision="no_timm")
```

Apply the same preprocessing that we did in the table detection.

```
[ ]: import numpy as np
import cv2
from PIL import Image
```

```

from torchvision import transforms

class MaxResize(object):
    def __init__(self, max_size=800):
        self.max_size = max_size

    def __call__(self, image):
        width, height = image.size
        current_max_size = max(width, height)
        scale = self.max_size / current_max_size
        resized_image = image.resize((int(round(scale * width)),
↪int(round(scale * height))), Image.Resampling.LANCZOS)
        return resized_image

class ToGrayscale(object):
    def __call__(self, img):
        return img.convert("L")

class ApplyThresholding(object):
    def __call__(self, img):
        img_cv = np.array(img)
        if len(img_cv.shape) == 3: # Convert to grayscale if it's not already
            img_cv = cv2.cvtColor(img_cv, cv2.COLOR_RGB2GRAY)
        _, img_thresh = cv2.threshold(img_cv, 0, 255, cv2.THRESH_BINARY + cv2.
↪THRESH_OTSU)
        return Image.fromarray(img_thresh)

class ApplyDilation(object):
    def __init__(self, kernel_size=3):
        self.kernel = np.ones((kernel_size, kernel_size), np.uint8)

    def __call__(self, img):
        img_cv = np.array(img)
        img_dilated = cv2.dilate(img_cv, self.kernel, iterations=1)
        return Image.fromarray(img_dilated)

class EnhanceContrast(object):
    def __call__(self, img):
        img_cv = np.array(img)
        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
        img_enhanced = clahe.apply(img_cv)
        return Image.fromarray(img_enhanced)

class EdgeDetection(object):
    def __call__(self, img):
        img_cv = np.array(img)
        edges = cv2.Canny(img_cv, 100, 200)

```

```

        return Image.fromarray(edges)

class GrayscaleTo3Channel(object):
    def __call__(self, img):
        img_array = np.array(img)
        if len(img_array.shape) == 2:  # If grayscale
            img_3ch = np.stack([img_array] * 3, axis=-1)
            return Image.fromarray(img_3ch)
        else:
            return img  # If already 3-channel, return as is

# Example usage of the transform pipeline
detection_transform = transforms.Compose([
    # ToGrayscale(),
    # ApplyThresholding(),
    # ApplyDilation(kernel_size=2),  # Adjust kernel_size as needed
    # GrayscaleTo3Channel(),
    MaxResize(800),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.485, 0.485], [0.229, 0.229, 0.229])  # Same
    ↪ mean and std for each channel
])

```

Retrieval Function for the Bboxes

```

[ ]: # for output bounding box post-processing
def box_cxxywh_to_xyxy(x):
    x_c, y_c, w, h = x.unbind(-1)
    b = [(x_c - 0.5 * w), (y_c - 0.5 * h), (x_c + 0.5 * w), (y_c + 0.5 * h)]
    return torch.stack(b, dim=1)

def rescale_bboxes(out_bbox, size):
    img_w, img_h = size
    b = box_cxxywh_to_xyxy(out_bbox)
    b = b * torch.tensor([img_w, img_h, img_w, img_h], dtype=torch.float32)
    return b

# update id2label to include "no object"
id2label = model.config.id2label
id2label[len(model.config.id2label)] = "no object"

def outputs_to_objects(outputs, img_size, id2label):
    m = outputs.logits.softmax(-1).max(-1)
    pred_labels = list(m.indices.detach().cpu().numpy())[0]

```

```

pred_scores = list(m.values.detach().cpu().numpy())[0]
pred_bboxes = outputs['pred_bboxes'].detach().cpu()[0]
pred_bboxes = [elem.tolist() for elem in rescale_bboxes(pred_bboxes,
↳img_size)]

objects = []
for label, score, bbox in zip(pred_labels, pred_scores, pred_bboxes):
    class_label = id2label[int(label)]
    if not class_label == 'no object':
        objects.append({'label': class_label, 'score': float(score),
                        'bbox': [float(elem) for elem in bbox]})

return objects

```

Finally, we will plot the columns to gain a visual of the model results.

```

[ ]: import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.patches import Patch

def fig2img(fig):
    """Convert a Matplotlib figure to a PIL Image and return it"""
    import io
    buf = io.BytesIO()
    fig.savefig(buf)
    buf.seek(0)
    img = Image.open(buf)
    return img

def visualize_detected_tables(img, det_tables, out_path=None):
    plt.imshow(img, interpolation="lanczos")
    fig = plt.gcf()
    fig.set_size_inches(20, 20)
    ax = plt.gca()

    for det_table in det_tables:
        bbox = det_table['bbox']

        if det_table['label'] == 'table column':
            facecolor = (1, 0, 0.45)
            edgecolor = (1, 0, 0.45)
            alpha = 0.3
            linewidth = 2
            hatch='////////'
        # if det_table['label'] == 'table row':
        #     facecolor = (0.95, 0.6, 0.1)

```

```

#     edgecolor = (0.95, 0.6, 0.1)
#     alpha = 0.3
#     linewidth = 2
#     hatch='////////'
else:
    continue

    rect = patches.Rectangle(bbox[:2], bbox[2]-bbox[0], bbox[3]-bbox[1],
↪linewidth=linewidth,
                                edgecolor='none',facecolor=facecolor,
↪alpha=0.1)
    ax.add_patch(rect)
    rect = patches.Rectangle(bbox[:2], bbox[2]-bbox[0], bbox[3]-bbox[1],
↪linewidth=linewidth,
                                edgecolor=edgecolor,facecolor='none',linestyle='-', alpha=alpha)
    ax.add_patch(rect)
    rect = patches.Rectangle(bbox[:2], bbox[2]-bbox[0], bbox[3]-bbox[1],
↪linewidth=0,
                                edgecolor=edgecolor,facecolor='none',linestyle='-', hatch=hatch, alpha=0.2)
    ax.add_patch(rect)

plt.xticks([], [])
plt.yticks([], [])

legend_elements = [Patch(facecolor=(1, 0, 0.45), edgecolor=(1, 0, 0.45),
↪label='Table Column', hatch='////////', alpha=0.
↪3),
                                Patch(facecolor=(0.95, 0.6, 0.1), edgecolor=(0.95, 0.6,
↪0.1),
                                label='Table Row', hatch='////////', alpha=0.3)]
plt.legend(handles=legend_elements, bbox_to_anchor=(0.5, -0.02), loc='upper_
↪center', borderaxespad=0,
                                fontsize=10, ncol=2)
plt.gcf().set_size_inches(10, 10)
plt.axis('off')

if out_path is not None:
    plt.savefig(out_path, bbox_inches='tight', dpi=150)

plt.clf()

return fig

```

```
[ ]: def runModel(files):
    file_path = []
    score_list = []
    for path in files:
        image = Image.open(path).convert("RGB")
        pixelval = detection_transform(image).unsqueeze(0)
        pixelval = pixelval.to(device)
        with torch.no_grad():
            outputs = model(pixelval)
            objects = outputs_to_objects(outputs, image.size, id2label)
            file_path.append(path)
            score_list.append(objects)
    data = pd.DataFrame({"file_path":file_path, "score": score_list})
    data.to_csv("columnSample.csv", index=False)
    return data
```

```
[ ]: mydf = runModel(table_paths)
```

```
[ ]: def markTable(index, data):
    image_path = data["file_path"][index]
    object = data["score"][index]
    image = Image.open(image_path).convert("RGB")
    visualize_detected_tables(image, object, out_path=f"{image_path[:
↵-4]}"+ "_columns.png")
```

```
[ ]: for i in range(mydf.shape[0]):
    markTable(i, mydf)
```

<Figure size 1000x1000 with 0 Axes>

2 Select Examples

Clear and distinct tables results in clean and desirable columns, below is an example for document 24284 page 20.

```
[ ]: image = Image.open('tables/24284_page20.png/table_rotated_0_columns.png')
    image
```

```
[ ]:
```

Measurement site number	Classi- fication	Elevation of measurement site (feet)	Median measured discharge (gpm)	P. ₁₀ (gpm)	P. ₉₀ (gpm)	Ratio $\frac{P_{10}}{P_{90}}$
1	2	3	4	5	6	7
2	Spring	2,700	5.4	17	1.2	14
18	do	1,900	7.2	10	3.2	3
*20 & 21	do	1,700	7.2	13	3.7	3.5
28	do	1,250	12	37	3.3	11
31	do	425	39	94	28	3.5
33	Stream	500	33	64	15	4
35	Spring	800	54	158	25	6
36	do	600	3.2	15	1.3	12
49	Stream	425	4.9	28	.51	55
50	do	520	425	1,410	149	9.5
59	do	1,100	58	331	1.0	331
61	do	190	31	90	7.6	12
65	do	800	162	830	21	40
70	Spring	100	94	630	2.7	233
85	Stream	400	111	260	42	6
93	Spring	2,320	6.7	15	3.1	5
94	do	2,350	1.4	2.7	.6	4.5
100	do	2,025	3.8	34	.32	106
106a	do	1,100	.75	2.5	.29	8.5
110a	Stream	950	28	176	4.5	39
122	do	1,600	6.7	45	1.1	41
132	Spring	1,250	2.6	10	.43	23
136	do	970	1.5	6.4	.36	18
140	do	1,150	47	262	6.2	42
142	do	980	.15	.41	.001	410
143	Stream	2,200	45	130	17	7.5
147	Spring	1,750	.85	1.25	.40	3
153	do	1,150	.56	2.0	.044	45

Table Column Table Row

The main issue comes with cluttered tables. Tables with a lot of stacked numbers create a lot of noise. Below is an example of the noisy bounding box. Document 2344 page 41.

```
[ ]: Image.open("tables/2344_page40.png/table_0_columns.png")
```

```
[ ]:
```


TABLE 9.—Chemical analyses, in parts per milli

Well	Owner	Date sampled	Temp (° F)	Specific conductance (micro mhos at 25° C)	Silica (SiO ₂)	Calcium (Ca)	Magnesium (Mg)	Sodium (Na)	Fat (l)
8/32-18P1	James Hopkins	12- 9-43	65	2,300					
28R1	Bernice Holt	12- 8-43		1,420					
29E1	Coast Counties Warehouse	12- 9-43		1,140					
29N1	Santa Barbara County	12- 9-43		532					
29R2	Raffoni	12- 8-43		960					
30C2	James Hopkins	12- 4-43		539					
30E1	John Parma	12- 8-43		457					
30G1	James Hopkins	5-28-58		848	50	65	30	58	2
30K1	John Parma	12- 8-43	62	807					
30K2	do	12- 9-43	61	780					
30Q1	Conaglia Brothers	12- 8-43		599					
30Q1	Frank F. Barham	12- 8-43		553					
8/33-14R1	Tognazzini Brothers	12- 9-43		646					
19E1	Careaga School	11-26-27		1,240	8	91	26	122	
19Q1	E. M. Carr	12- 8-43		1,600					
19H1	Careaga Estate	12- 9-43		1,970					
19L3	A. B. Hanson	12- 8-43		1,380					
19R1	Tony Careaga	12- 9-43		1,270					
20K1	Manuel Luis	12- 8-43		1,000					
20M1	Walter Barco	12- 8-43		1,850					
20Q1	Virginia Barco Estate	11-26-27		1,065	7	86	25	78	
20R1	do	9-26-57		1,039	50	73	54	79	2
Do	do	5- 7-58		1,200	20	114	44	93	2
Do	do	9-17-58		1,283	43	113	51	93	2
21G1	Carl Jensen	12- 9-43		916					
21M1	Shell Company of Calif.	11-26-27		1,185	7	96	40	74	
22N1	Luis Scolari	12- 8-43		824					
23P1	James Richard	12- 8-43		665					
24R1	James Hopkins	12- 9-43	50.5	814					
25B2	Carl Abele	12- 8-43		649					
25D3	James Hopkins	12- 9-43		523					
29D1	Tony Careaga	12- 9-43		1,180					
8/34-17B1	U.S. Government	12- 8-43	63	730					
17K1	do	5-28-58		5,240	50	36	31	1,025	5
22G1	Virginia Barco Estate	12- 8-43		1,420					
21R1	Joséphine Harris Estate	12- 8-43	61.5	1,210					

Table Column Table Row

2.1 Other Options: CV2 Line Detection

```
[ ]: # Load image, convert to grayscale, Otsu's threshold
image = cv2.imread('sampleTables/1166_page31.png')
result = image.copy()
gray = cv2.cvtColor(image,cv2.COLOR_BGR2GRAY)
thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)[1]

# Detect horizontal lines
```



```

horizontal_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (150,1))
detect_horizontal = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, horizontal_kernel,
    ↪iterations=2)
cnts = cv2.findContours(detect_horizontal, cv2.RETR_EXTERNAL, cv2.
    ↪CHAIN_APPROX_SIMPLE)
cnts = cnts[0] if len(cnts) == 2 else cnts[1]
for c in cnts:
    cv2.drawContours(result, [c], -1, (36,255,12), 2)

# Detect vertical lines
vertical_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (1,6))
detect_vertical = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, vertical_kernel,
    ↪iterations=2)
cnts = cv2.findContours(detect_vertical, cv2.RETR_EXTERNAL, cv2.
    ↪CHAIN_APPROX_SIMPLE)
cnts = cnts[0] if len(cnts) == 2 else cnts[1]
for c in cnts:
    cv2.drawContours(result, [c], -1, (36,255,12), 2)

cv2.imwrite("cv2LineDetection.png", result)

```

[]: True

[]: Image.open("cv2LineDetection.png")

[]:

24 STORAGE RESERVOIRS ON STONY CREEK, CALIFORNIA. [NO. 86.]

Precipitation, in inches, at Little Stony, Colusa County.

[Latitude, 39° 25'; longitude, 122° 30'; authority, C. M. Polley.]

Year.	Sept.	Oct.	Nov.	Dec.	Jan.	Feb.	Mar.	Apr.	May.	June.	July.	Aug.	Total.
1884-85.....				8.40	1.29	1.04	0	1.38	0	1.19	0	0	^a 13.30
1885-86.....	0	0.27	13.29	3.51	3.44	0	1.10	3.89					^a 25.50

^a Year incomplete.

Precipitation, in inches, at Julian's ranch, on Stony Creek, Glenn County.

[Latitude, 39° 35'; longitude, 122° 30'; elevation, 750 feet; authority, Mrs. Lee Julian.]

Year.	Sept.	Oct.	Nov.	Dec.	Jan.	Feb.	Mar.	Apr.	May.	June.	July.	Aug.	Total.
1900-1901.....						4.26	0.12	1.24	0.50	0	0	0	^a 6.12
1901-2.....	1.07	0.60	3.18	0.82	0.85	9.27	3.24	2.37	2.06	0	0	0	23.46

^a Year incomplete.

Precipitation, in inches, at Camp Wright, Mendocino County.

[Latitude, 39° 45'; longitude, 123°; elevation, 1,800 feet; authority, U. S. War Department.]

Year.	Sept.	Oct.	Nov.	Dec.	Jan.	Feb.	Mar.	Apr.	May.	June.	July.	Aug.	Total.
1864-65.....	0.03	0.85	6.19	12.30	3.64	4.88	3.57	0.36	0.43	0	0	0	32.15
1865-66.....	.81	.87	16.67	2.06	15.83	4.30	11.80	1.05					^a 53.38
1866-67.....	0	0	5.92	24.67	8.92	7.85	2.38	3.10	.01	0	0	0	52.85
1867-68.....	1.56	3.75	6.38	29.03	8.61	5.89	9.11	3.25	.71	1.58	0	0	69.87
1868-69.....	.05	.25	2.79	6.63	13.57	4.13	3.80	4.51	.55	0	0	0	36.28
1869-70.....	1.42	.84	5.38	8.06	7.03	6.61	2.40	2.02	.39	0	0	0	34.15
1870-71.....	0	0	1.28	1.19	2.66	4.60	7.10	1.07	1.96	0	0	0	19.86
1871-72.....	.50	.31	4.03	16.64	11.52	19.78	5.34	.66	.12	.04	0	0.20	59.14
1872-73.....	0	.36	5.20	7.24	3.55	6.92	2.91	1.13	.04	.30	0.10	0	27.75
1873-74.....	.05	.34	4.98	15.50	12.94	5.46	7.26	3.72	1.16	.26	.01	0	51.68
1874-75.....	0	5.61	12.99	2.27	9.83	1.06	1.76	.59	2.32	0	0	0	36.43
Mean.....	.40	1.20	6.53	11.41	8.92	6.50	5.22	1.95	.77	.20	.01	.02	43.13

^a Year incomplete.